Динамическое программирование

Динамическое программирование

Цель: освоение навыков решения оптимизационных задач с использованием динамического программирования

Задачи:

- изучение теоретических основ динамического программирования;
- освоить практическое применение динамического программирования для решения оптимизационных задач.

План лекции

1. Теоретические основы динамического программирования
2. Теоретические основы рекурсивных алгоритмов;
3. Решение задачи о рюкзаке;
4. Задача о вычислении дистанции Левенштейна;
5. Решение задачи о расстановке скобок при перемножении матриц;
6. Решение задачи вычисления длины наибольшей общей подпоследовательности;

Понятие о динамическом программировании

Динамическое программирование используется при оптимальном планировании управляемых процессов и наиболее эффективно в случае многошаговых или многоэтапных процессов принятия решений.

Понятие о динамическом программировании

Анализируется некоторая управляемая система, в которой происходят экономические, производственные, технологические или иные многошаговые или многоэтапные процессы. Для каждого из допустимых управлений задается показатель эффективности управления (целевая функция). В экономических системах показатель эффективности может представлять прибыль, затраты, рентабельность, объем производства и т.п.

Задача ДП состоит в поиске оптимального управления, переводящего систему из начального состояния в конечное, и обеспечивающего экстремум целевой функции.

Понятие о динамическом программировании

Динамическое программирование позволяет свести глобальную оптимизацию аддитивной или мультипликативной целевой функции к поэтапной оптимизации промежуточных целевых функций.

Аддитивная ЦФ может быть представлена в виде

$$f\left(x_1, x_2, \ldots, x_n\right) = \sum_{j=1}^{n} f_j\left(x_j\right) ,$$

где слагаемые соответствуют эффектам решений, принимаемых на отдельных этапах управляемого процесса.

Мультипликативная функция представляет произведение “одношаговых” функций:

$$f\left(x_1, x_2, \ldots, x_n\right) = \prod_{j=1}^{n} f_j(x_j) .$$

Понятие о динамическом программировании

К преимуществам метода динамического программирования по сравнению с “классическими” методами оптимизации относятся более высокая скорость расчетов и широкая область применимости. В частности, для него некритично требование линейности и дифференцируемости функций $\mathbf{f_j}(\mathbf{x_j})$ и функция выигрыша может быть задана не в аналитическом, а в табличном виде. Другими словами, метод применим и при решении задач нелинейного и дискретного программирования.

Понятие о динамическом программировании

Классическим примером задачи ДП является планирование промышленного объединения, состоящего из ***k*** предприятий на период ***n*** лет. Выделяемые в начале каждого года средства должны быть распределены между предприятиями таким образом, чтобы суммарный доход за весь период планирования был максимальным.

Понятие о динамическом программировании

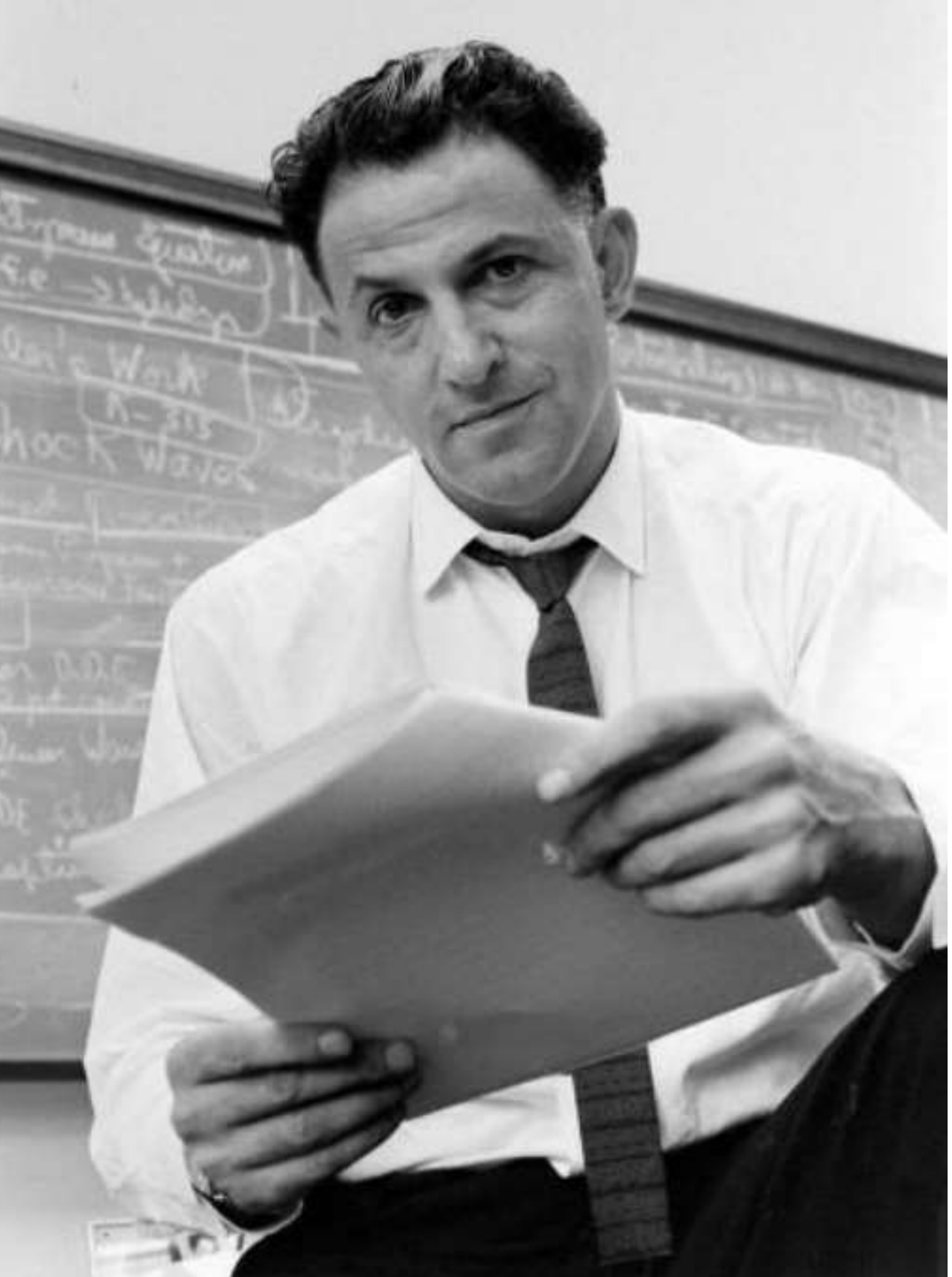
Общий доход $\boldsymbol{W}$ равен сумме доходов на отдельных шагах (годах)

$$W = \sum_{i=1}^{n} w_i \, .$$

Шаговое управление $\boldsymbol{u_i}$ заключается в выделении в начале года $\boldsymbol{i}$ предприятиям средств $\boldsymbol{u_{i1}}$, $\boldsymbol{u_{i2}}$, … , $\boldsymbol{u_{ik}}$ (первый индекс – номер шага (года), второй – номер предприятия), т.е. представляет вектор с $\boldsymbol{k}$ составляющими

$$u_i = (u_{i1,} u_{i2,} \ldots., u_{ik})$$

Задача сводится к поиску такого распределения средств по предприятиям и годам (оптимального управления $\boldsymbol{u^*}$), чтобы получаемая суммарная прибыль была максимальна.



Ричард Эрнст Беллман (*Richard Ernest Bellman*; 1920—1984) — американский математик, один из ведущих специалистов в области математики и вычислительной техники

Понятие о динамическом программировании

Принцип оптимальности Беллмана и алгоритм решения задач динамического программирования

Принципиально важно, что метод динамического программирования не сводится к простой оптимизации каждого шага управления независимо от других шагов. Выбор шагового управления проводится с учетом будущих последствий принимаемых решений.

Это положение конкретизируется в принципе оптимальности Беллмана, составляющем основу метода динамического программирования:

Каково бы ни было состояние управляемой системы S перед очередным шагом, шаговое управление необходимо выбирать так, чтобы выигрыш на данном шаге плюс оптимальный выигрыш на всех последующих шагах был максимальным.

Понятие о динамическом программировании

Принцип оптимальности Беллмана и алгоритм решения задач динамического программирования

Так как среди всех шагов есть один, который можно планировать без учета его последствий (последний шаг ***n***), то процесс динамического программирования обычно проводится в направлении от конца к началу.

Последний шаг планируется с учетом различных предположений о том, как закончился предпоследний (***n*-1**)-й шаг, и для каждого из этих предположений находится условное оптимальное управление на ***n*** -м шаге.

Процесс продолжается, т.е. проводится оптимизация управления на предпоследнем (***n*-1**)-м шаге с учетом всех возможных предположений об окончании (***n*-2**)-го шага и т.д. вплоть до первого шага.

После определения условно оптимальных управлений на всех шагах определяется оптимальное управление для всего процесса.

Алгоритм реализации метода ДП

1. Определяются ***параметры состояния*** управляемой системы и ***набор шаговых управлений*** u_i ;

2. определяется ***выигрыш на i-м шаге*** управления, если перед этим система находилась в состоянии (S_{i-1})

(***функция выигрыша***):

$$w_i = f_i(S_{i-1}, u_i) \qquad (4)$$

3. определяется ***изменение состояния системы*** при управлении u_i

$$S_i = \varphi_i(S_{i-1}, u_i) \qquad (5)$$

(***переход системы*** из состояния $i-1$ в состояние i; при этом должны быть известны «функции изменения состояния» (5));

4. Проводится ***условная оптимизация последнего*** (n-го)шага

$$W_n(S_{n-1}) = \max_{u_n}\{f_n(S_{n-1}, u_n)\} \quad ; \qquad (6)$$

5. Проводится условная оптимизация $(n-1)$-го, $(n-2)$-го, и т.д. шагов с помощью ***рекуррентного соотношения ДП*** (***уравнения Беллмана***)

$$W_i(S_{i-1}) = \max_{u_i}\{f_i(S_{i-1}, u_i) + W_{i+1}(\varphi_i(S_{i-1}, u_i))\} \ , \quad (7)$$

в котором последовательно используются значения $i = n-1,\ n-2$ и т.д. – это позволяет рассчитать ***условный оптимальный выигрыш*** $W_i(S_{i-1})$ (начиная с i-го шага и ***до конца***)

6. Производится ***безусловная*** оптимизация путем перемещения от первого шага к последнему.

Рекурсивные алгоритмы

Многие оптимизационные алгоритмы основаны на принципе разбиения основной задачи на подзадачи, каждая из которых повторяет основную, но входные их данные таковы, что область допустимых решений становится меньше.

Рекурсивный алгоритм – это алгоритм, решающий задачу путем сведения ее к решению одной или нескольких таких же задач, но в сокращенном их варианте.

Неразрывно с понятием рекурсивного алгоритма связано понятие **рекурсивной функции**.

Рекурсивные алгоритмы

Существует два определения понятия рекурсивной функции .

Первое определение рекурсивной функции относится к теории вычислимости и является синонимом понятия вычислимой функции, т. е. функции, для вычисления значения которой можно указать алгоритм.

Второе определение, которое и будет использоваться здесь, происходит из области теории программирования.

Рекурсивная функция – это функция, которая вызывает саму себя.

Рекурсивные алгоритмы

Рекурсивный алгоритм может быть записан в виде рекурсивной функции.

Классическими примерами рекурсивных функций являются функции для вычисления факториала, чисел Фибоначчи и наибольшего общего делителя с помощью алгоритма Эвклида.

Рекурсивную функцию всегда можно преобразовать в цикл, и, наоборот любой цикл можно представить в виде рекурсивной функции.

```
//- Рекурсивные функции
//-- вычисление факториала числа
 unsigned int fact(unsigned int x)
{ return (x == 0)?1:x*fact(x-1); };

 //-- вычисление наибольшего общего делителя
 unsigned int gcd(int m, int n)   // Эвклид
     { return (n == 0)?m:gcd(n, m%n); };

 // -- вычисление  n-го члена ряда Фибоначчи(1170-1250)
unsigned int fib(int n)
{ return (n < 1)?0:((n == 1)?1:fib(n-1)+fib(n-2));};
```

```
//- Вычисление факториала числа при помощи цикла
 unsigned int fact(unsigned int x)
  {
            int rc = 1;
            for (int i = 1; i<=x; i++) rc *= i;
            return rc;
   };
```

Рекурсивные алгоритмы

Рекурсивная запись алгоритма, как правило, не дает выигрыша в скорости его работы. Скорее наоборот, так как вызов любой функции связан с сохранением и восстановлением контекста вызывающей функции, что является затратной по времени операцией.

Кроме того, для хранения контекста операционной системой резервируется специальная секция памяти, называемая системным стеком.

Рекурсивные алгоритмы

Если цепочка вызовов функций является длинной (иногда говорят о большой ***глубине рекурсии***), то это может привести к переполнению стека. Например, при вычислении факториала числа 25 глубина рекурсии достигает значения 24.

Часто рекурсивные функции, применяемые для решения оптимизационных задач, используют более одного рекурсивного вызова, каждый из которых работает приблизительно с половиной входных данных. Такую схему решения называют ***«разделяй и властвуй»***.

Динамическое программирование

Решение задачи о рюкзаке

Метод решения задачи оптимизации, реализующей рекурсивный алгоритм с перекрывающимися подзадачами, в котором каждая такая подзадача решается один раз, а ее результат сохраняется для последующего применения, называется динамическим программированием.

Рассматриваемый здесь метод является частью более общей теории динамического программирования, основы которой разработаны Р. Беллманом. Эта теория исследует процесс пошагового решения задач оптимизации, в котором на каждом шаге из множества допустимых решений выбирается одно, оптимизирующее заданную целевую функцию.

Решение задачи о рюкзаке

Классическая формулировка задачи допускает выбор одинаковых предметов при размещении их в рюкзаке. Математическая модель классической задачи о рюкзаке выглядит следующим образом:

$$\sum_{i=1}^{n} x_i v_i c_i \to \max, \ \sum_{i=1}^{n} x_i v_i \le V, \ x_i \in \left\{0, 1, 2, \ldots, \left\lfloor \frac{V}{v_i} \right\rfloor\right\}, \ i = \overline{1, n},$$

где x_i – неизвестные (количество предметов каждого типа), которые требуется найти; v_i – объемы предметов; c_i– стоимость единицы объема каждого предмета; V – вместимость рюкзака;.

Решением задачи при такой постановке будет вектор $X = (x_1, x_2, \ldots, x_n)$. Каждый элемент x_i вектора может принимать целое значение из отрезка $\left[0, \left\lfloor \frac{V}{v_i} \right\rfloor\right]$. При этом, если $x_i = 0$, то i-й предмет не выбран, и если $x_i > 0$, то i-х предметов в рюкзак помещено x_i единиц.

Последовательность рекуррентных соотношений, позволяющая вычислить значение максимальной стоимости рюкзака равна

$$Z = Z_{1,n} = \max_{x_n}\left(Z_{1,n-1} + x_n v_n c_n\right),$$

$$x_n \in \left\{0, 1, \ldots, \left\lfloor \frac{V}{v_n} \right\rfloor\right\},$$

$$Z_{1,n-1} = \max_{x_{n-1}}\left(Z_{1,n-2} + x_{n-1} v_{n-1} c_{n-1}\right),$$

$$x_{n-1} \in \left\{0, 1, \ldots, \left\lfloor \frac{V - R_{n,n}}{v_{n-1}} \right\rfloor\right\},$$

$$Z_{1,n-2} = \max_{x_{n-2}}\left(Z_{1,n-3} + x_{n-2} v_{n-2} c_{n-2}\right),$$

$$x_{n-2} \in \left\{0, 1, \ldots, \left\lfloor \frac{V - R_{n-1,n}}{v_{n-2}} \right\rfloor\right\}, \ldots, \quad Z_{1,1} = x_1 v_1 c_1, \quad x_1 = \left\lfloor \frac{V - R_{2,n}}{v_1} \right\rfloor$$

Схема рекурсивного решения задачи о рюкзаке

$$n = 3,\ V = 120,\ v_1 = 30,\ v_2 = 50,\ v_3 = 20,\ c_1 = 5,\ c_2 = 9,\ c_3 = 3$$

Все вершины дерева, кроме корневой, изображают этапы решения. Вершины помечены двумя числами: первое число – текущая стоимость рюкзака, второе – остаток неиспользованного объема рюкзака. Все этапы образуют три слоя, что определяет глубину рекурсии. Каждой вершине (этапу) соответствует вызов рекурсивной функции.

Вершины соединены дугами, указывающими связь между этапами решения. Каждая дуга имеет метку, обозначающую предположение, при котором решается очередной этап.

Схема рекурсивного решения задачи о рюкзаке

Для примера, рассмотрим маршрут, образованный дугами с метками

$\boldsymbol{x}_3=2$, $\boldsymbol{x}_2=1$ и $\boldsymbol{x}_1=1$.

Первый этап в этом маршруте отмечен меткой 120, 80, означающей, что при размещении в рюкзаке двух предметов с номером 3 стоимость рюкзака станет 2·20·3=120 единиц, и при этом в рюкзаке останется 120 - 2·20=80 единиц объема.

Второй этап имеет метку 570, 30. Решение на этом этапе осуществляется в предположении, что в рюкзаке два предмета с номером 3 и один предмет с номером 2. Поэтому 570=120+1·50·9 и 30=80-1·50.

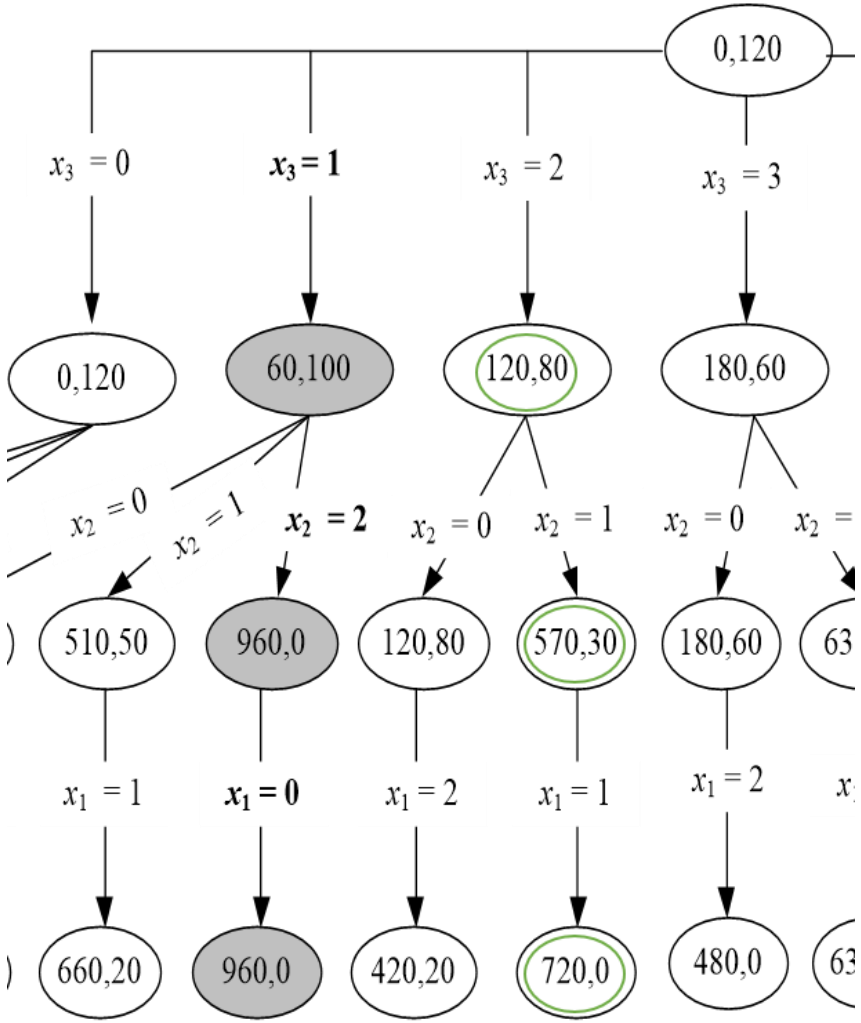


Схема рекурсивного решения задачи о рюкзаке

На третьем этапе завершается формирование одного из допустимых решений. Неиспользованный остаток объема в 30 единиц позволяет поместить только один предмет первого типа. В окончательном решении, соответствующем этому маршруту, стоимость предметов, уложенных в рюкзак, равна 570+1·30·5=720

Несложно заметить, что разобранный маршрут не соответствует оптимальному решению. Оптимальным будет решение ***X***=(0, 2, 1), а соответствующая ему стоимость – 960. Этапы этого решения обозначены закрашенными овалами.

Из схемы видно, что в рекурсивном решении, как и в случае с использованием генератора, осуществляется полный перебор допустимых решений.

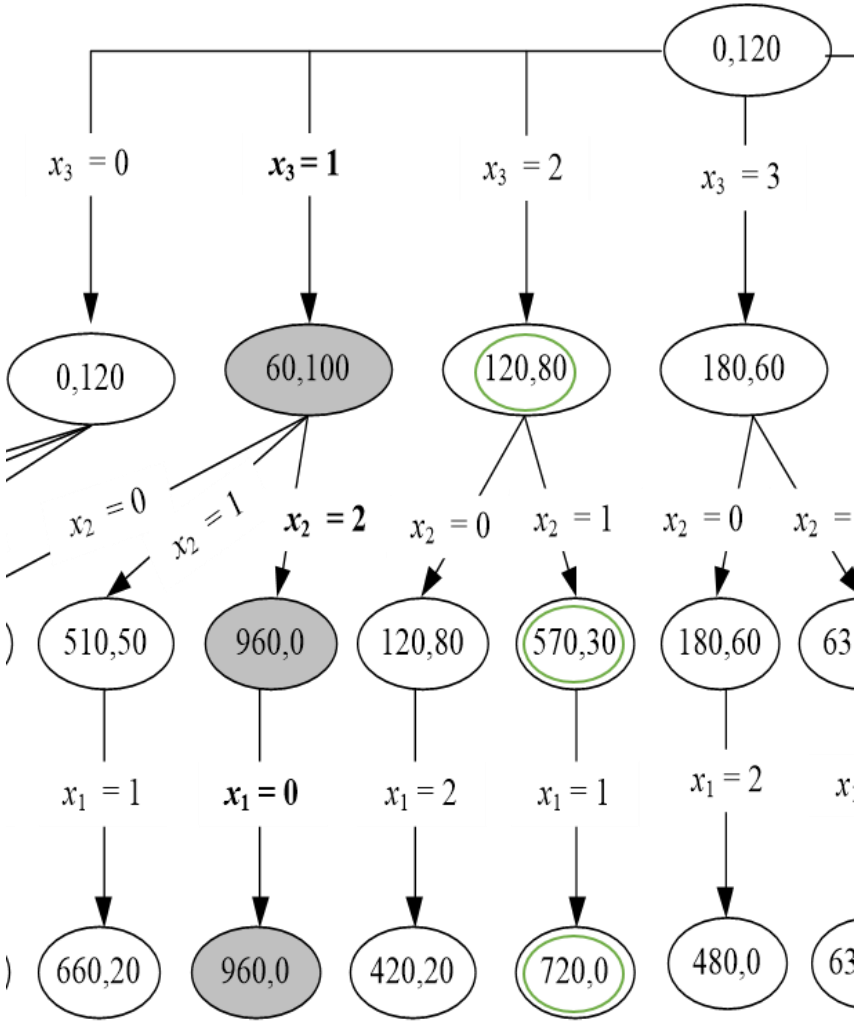
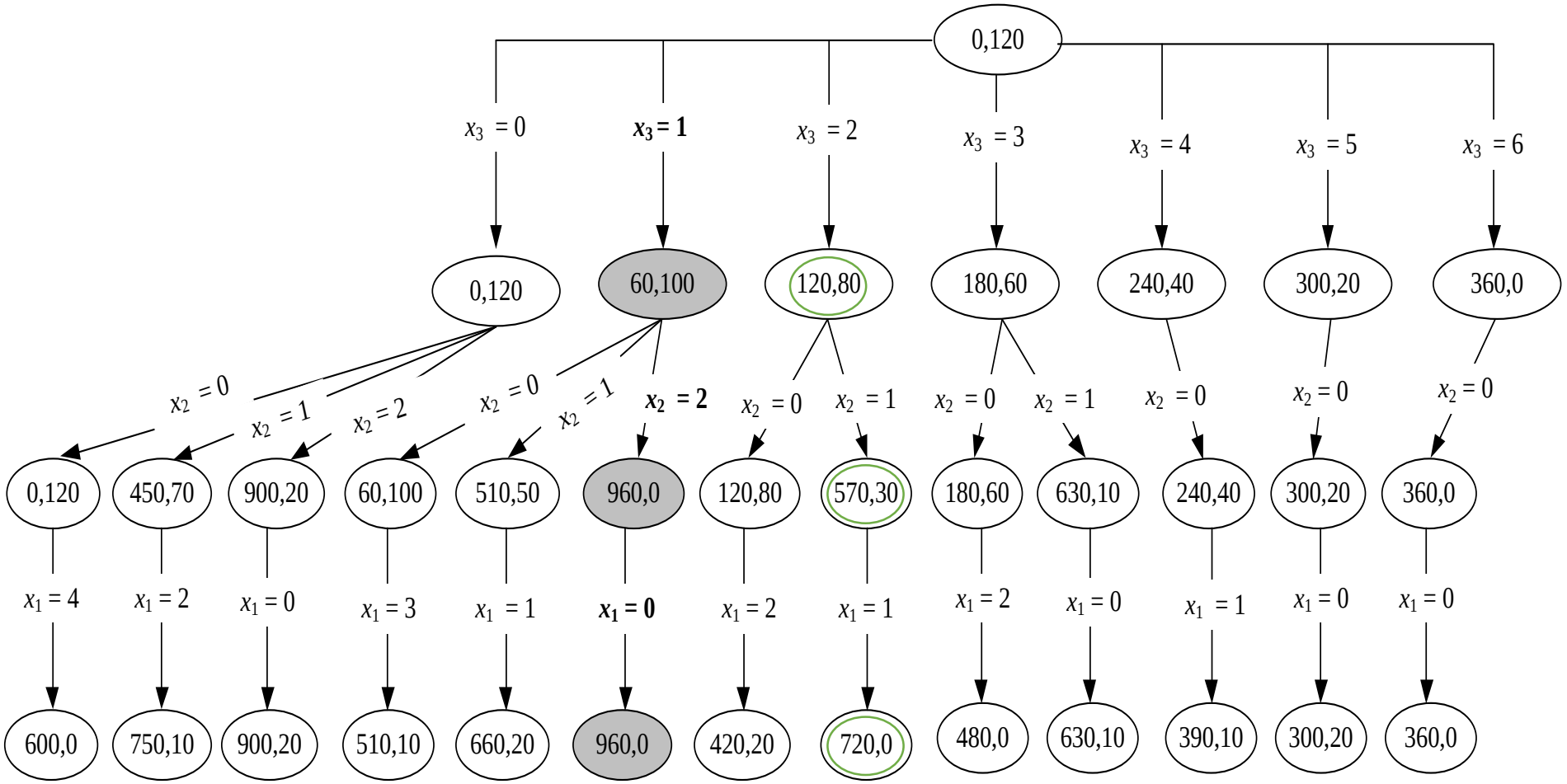


Схема рекурсивного решения задачи о рюкзаке

$n = 3$, $V = 120$, $v_1 = 30$, $v_2 = 50$, $v_3 = 20$, $c_1 = 5$, $c_2 = 9$, $c_3 = 3$



Решение задачи о рюкзаке

Применение динамического программирования при решении оптимизационных задач обычно предполагает создание специальных таблиц для хранения промежуточных результатов.

Изображены таблицы, которые используются для решения задачи о рюкзаке методом динамического программирования. Векторы, определяющие размеры(v) и стоимости (c) типов предметов, а также величина, характеризующая вместимость рюкзака (V), заданы в верхней части рисунка.

$V = 300$

v	85	50	75	65	30	40	30
c	150	25	30	40	40	35	45

S

300	0	0

T.0

300	0	0
300	85	12 750
300	170	25 500
300	255	38 250

T.1

300	0	0
300	50	1 250
300	100	2 500
300	150	3 750
300	200	5 000
300	250	6 250
300	300	7 500
215	0	12 750
215	50	14 000
215	100	15 250
215	150	16 500
215	200	17 750
130	0	25 500
130	50	26 750
130	100	28 000
45	0	38 250

T.2

300	0	0
300	75	2 250
300	150	4 500
300	225	6 750
300	300	9 000
250	0	1 250
250	75	3 500
250	150	5 750
250	225	8 000
200	0	2 500
200	75	4 750
200	150	7 000
150	0	3 750
150	75	6 000
150	150	8 250
100	0	5 000
100	75	7 250
50	0	6 250
0	0	7 500
215	0	12 750
215	75	15 000
215	150	17 250
165	0	14 000
165	75	16 200
165	150	18 500
115	0	15 250
115	75	17 500
65	0	16 500
15	0	17 750
130	0	25 500
130	75	27 500
80	0	26 750
80	75	29 000
30	0	28 000

T.3

300	0	0
300	65	2 600
...	...	...
300	260	10 400
225	0	2 250
...	...	...
225	195	10 050
150	0	4 500
150	65	7 100
150	130	9 700
75	0	6 750
75	65	9 350
0	0	9 000
250	0	1 250
...	...	...
250	195	9 050
175	0	3 500
175	65	6 100
175	130	8 700
100	0	5 750
100	65	8 350
200	0	2 500
...	...	...
200	195	10 300
125	0	4 750
125	65	7 350
115	75	17 500
50	0	7 000
150	0	3 750
150	65	6 350
150	130	8 950
75	0	6 000
75	65	8 600
0	0	8 250

T.4

300	0	0
300	30	1 200
...	...	...
...	...	...
...	...	...
30	0	28 000
30	30	29 200
45	0	38 250
45	30	39 450

T.5

300	0	0
300	40	1 400
...	...	...
...	...	...
...	...	...
30	0	28 000
45	0	38 250
45	40	39 650
15	0	39 450

T.6

300	0	0
300	30	1 350
...	...	...
...	...	...
...	...	...
30	0	28 000
30	30	39 600
45	30	39 600
5	0	39 650

Владимир Иосифович Левенштейн — советский и российский математик, доктор физико-математических наук.

ДИСТАНЦИЯ ЛЕВЕНШТЕЙНА

Дистанция Левенштейна (расстояние Левенштейна, редакционное расстояние, дистанция редактирования) определяется между двумя строками и равна минимальному количеству операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Расстояние Левенштейна активно применяется для исправления ошибок в поисковых системах, в текстовых редакторах, а также в биоинформатике.

Вычисление дистанции Левенштейна

Пусть X и Y– две символьные строки, тогда для вычисления дистанции Левенштейна L(X,Y) между ними может быть использовано следующее рекуррентное соотношение:

$$L(X,Y)=\begin{cases} len(Y),\ len(X)=0, \\ len(X),\ len(Y)=0, \\ 0,\ len(X)=len(Y)=1\wedge x=y, \\ 1,\ len(X)=len(Y)=1\wedge x\neq y, \\ \min\begin{cases} L(cut(X),Y)+1, \\ L(X,cut(Y))+1, \\ \begin{cases} L(cut(X),cut(Y)),\ last(X)=last(Y), \\ L(cut(X),cut(Y))+1,\ last(X)\neq last(Y). \end{cases} \end{cases} \end{cases}$$

- *len(X)* – количество символов в заданной строке. Например, *len(“привет”)*=6
- *cut(X)* – заданная строка без последнего символа. Например, *cut(“привет”)= “приве”*
- *last(X)* – последний символ заданной строки. Например, *last(“привет”)=“т”*

$$L(\text{"сор"},\text{"спорт"}) = \ ?$$

1. $$L(\text{"сор"},\text{"спорт"}) = \min\begin{cases} L(\text{"со"},\text{"спорт"})+1, \\ L(\text{"сор"},\text{"спор"})+1, \\ L(\text{"со"},\text{"спор"})+1. \end{cases}$$

2. $$L(\text{"со"},\text{"спорт"}) = \min\begin{cases} L(\text{"с"},\text{"спорт"})+1, \\ L(\text{"со"},\text{"спор"})+1, \\ L(\text{"с"},\text{"спор"})+1. \end{cases}$$

3. $$L(\text{"сор"},\text{"спор"}) = \min\begin{cases} L(\text{"со"},\text{"спор"})+1, \\ L(\text{"сор"},\text{"спо"})+1, \\ L(\text{"со"},\text{"спо"}) \end{cases}$$

4. $$L("co","спор") = \min\begin{cases} L("c","спор")+1, \\ L("co","спо")+1, \\ L("c","спо")+1. \end{cases}$$

5. $$L("c","спорт") = \min\begin{cases} L("","спорт")+1, \\ L("c","спор")+1, \\ L("","спор")+1, \end{cases}$$

$$L("","спорт") = 5,$$
$$L("","спор") = 4.$$

Шаги вычисления с 1 по 14 соответствуют рекурсивному погружению, а шаги с 15 по 28 – восходящему вычислению.

Нетрудно убедиться, что для превращения слова «сор» в слово «спорт» достаточно удалить (или вставить) две буквы.

6. $$L("c","cnop") = \min\begin{cases} L("","cnop")+1, \\ L("c","cno")+1, \\ L("","cno")+1, \end{cases}$$

$$L("","cnop") = 4,\ L("","cno") = 3.$$

7. $$L("cop","cno") = \min\begin{cases} L("co","cno")+1, \\ L("cop","cn")+1, \\ L("co","cn")+1. \end{cases}$$

8. $$L("co","cno") = \min\begin{cases} L("c","cno")+1, \\ L("co","cn")+1, \\ L("c","cn"). \end{cases}$$ +0

9. $$L("cop","cn") = \min\begin{cases} L("co","cn")+1, \\ L("cop","c")+1, \\ L("co","c")+1. \end{cases}$$

10. $$L("cop","c") = \min\begin{cases} L("cop","") + 1, \\ L("co","c") + 1, \\ L("co","") + 1, \end{cases}$$

$L("cop","") = 3,\ L("co","") = 2.$

11. $$L("c","cno") = \min\begin{cases} L("","cno") + 1, \\ L("c","cn") + 1, \\ L("","cn") + 1, \end{cases}$$

$L("","cno") = 3,\ L("","cn") = 2.$

12. $$L("co","cn") = \min\begin{cases} L("c","cn") + 1, \\ L("co","c") + 1, \\ L("c","c") + 1, \end{cases}$$

$L("c","c") = 0.$

13. $$L("c","cn") = \min\begin{cases} L("","cn") + 1, \\ L("c","c") + 1, \\ L("","c") + 1, \end{cases}$$

$L("","cn") = 2,\ L("c","c") = 0,\ L("","c") = 1.$

14. $$L("co","c") = \min\begin{cases} L("c","c")+1, \\ L("co","")+1, \\ L("c","")+1, \end{cases}$$

$$L("c","c")=0,\ \ L("co","")=2,\ \ L("c","")=1.$$

15. $L("co","c") = \min(1,\ 3,\ 2) = 1.$
16. $L("c","cn") = \min(3,\ 1,\ 2) = 1.$
17. $L("co","cn") = \min(2,\ 2, 1) = 1.$
18. $L("c","cno") = \min(4,\ 2,\ 3) = 2.$
19. $L("cop","c") = \min(2,\ 4,\ 3) = 2.$
20. $L("cop","cn") = \min(2,\ 2,\ 2) = 2.$
21. $L("co","cno") = \min(3,\ 2, 1) = 1.$
22. $L("cop","cno") = \min(2,\ 3,\ 2) = 2.$
23. $L("c","cnop") = \min(5,\ 3,\ 4) = 3.$
24. $L("c","cnopm") = \min(6,\ 4,\ 5) = 4.$
25. $L("co","cnop") = \min(4,\ 2,\ 3) = 2.$
26. $L("cop","cnop") = \min(3,\ 3, 1) = 1.$
27. $L("co","cnopm") = \min(5,\ 3, 4) = 3.$
28. $L("cop","cnopm") = \min(4,\ 2,\ 3) = 2.$

```
// - Levenshtein.h
// -- дистанции   Левенштейна (рекурсия)
int levenshtein_r(
                int lx,          // длина строки x
                                  const char x[],   // строка длиной lx
                                  int ly,          // длина строки y
                                  const char y[]    // строка y
                );
```

```
// - Levenshtein.cpp
#include "stdafx.h"
#include <algorithm>
#include "Levenshtein.h"
int min3(int x1, int x2, int x3) // выбрать минимум из 3x
{ return std::min(std::min(x1,x2),x3); }
int levenshtein_r(
                int lx, const char x[],
                int ly, const char y[] )
{ int rc = 0;
 if       (lx == 0) rc = ly;
 else  if  (ly == 0) rc = lx;
 else  if  (lx == 1 && ly == 1 && x[0] == y[0]) rc = 0;
 else  if  (lx == 1 && ly == 1 && x[0] != y[0]) rc = 1;
 else  rc = min3(
            levenshtein_r(lx-1, x, ly,   y)+1,
                    levenshtein_r(lx,   x, ly-1, y)+1,
                            levenshtein_r(lx-1, x, ly-1, y)+(x[lx-1] == y[ly-1]?0:1)      );
 return rc; };
```

Решение задачи о расстановке скобок при перемножении матриц

Вычислим количество операций умножения, которые необходимо выполнить при перемножении трех матриц A_1, A_2 и A_3 размерностью соответственно 10×100, 100×5 и 5×50.

Заметим, что операция умножения матриц ассоциативна и следовательно, $A_1 \times A_2 \times A_3 \equiv (A_1 \times A_2) \times A_3 \equiv A_1 \times (A_2 \times A_3)$.

Несложно подсчитать, что количество операций умножения, которые необходимо выполнить при вычислении произведения $(A_1 \times A_2) \times A_3$ будет $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$, а при вычислении $A_1 \times (A_2 \times A_3)$ – $100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$, т.е. в 10 раз больше.

Решение задачи о расстановке скобок при перемножении матриц

Рассмотрим произведение n матриц A_1, A_2, A_3, …, A_n, имеющих размерности соответственно $p_0 \times p_1$, $p_1 \times p_2$, $p_2 \times p_3$, …, $p_{n-1} \times p_n$.

$m_{i,j}$ – минимальное возможное количество операций умножения при вычислении произведения $A_{i,j} = A_i \times A_{i+1} \times \ldots \times A_j$, $i \leq j$.

$m_{i,i+1} = p_{i-1}p_ip_{i+1}$

$m_{i,i} = 0$

Разобьем произведение $A_{i,j} = A_i \times A_{i+1} \times \ldots \times A_j$ следующим образом: $A_{i,j} = (A_{i,k}) \times (A_{k+1,j})$, где $i \leq k < j$. Индекс k будем называть точкой разрыва.

Рекуррентное соотношение, позволяющее вычислить минимальное количество операций умножения:

Возьмите последовательность матриц и разделите ее на две подпоследовательности.
Найдите минимальную стоимость умножения каждой подпоследовательности.
Сложите эти затраты вместе и добавьте стоимость умножения двух результирующих матриц.
Сделайте это для каждой возможной позиции, в которой последовательность матриц может быть разделена, и возьмите минимум для всех из них.

$$m_{i,j} = \begin{cases} 0, i = j, \\ p_{i-1} p_i p_{i+1}, j - i = 1, \\ \min\limits_{i \le k < j} (m_{i,k} + m_{k+1,j} + p_{i-1} p_k p_j), j - i > 1. \end{cases}$$

Определим минимальное количество операций умножения для матриц A_1, A_2, *и* A_3, из рассмотренного выше примера:

$$m_{1,3} = \min_{1 \le k < 3} (m_{1,k} + m_{k+1,3} + p_0 p_k p_3);$$

$$k = 1,\ m_{1,1} + m_{2,3} + p_0 p_1 p_3 = 0 + p_1 p_2 p_3 + p_0 p_1 p_3 =$$

$$= 0 + 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75\,000;$$

$$k = 2,\ m_{1,2} + m_{3,3} + p_0 p_2 p_3 = p_0 p_1 p_2 + 0 + p_0 p_2 p_3 =$$

$$= 10 \times 100 \times 5 + 0 + 10 \times 5 \times 50 = 7500;$$

$$m_{1,3} = \min(75\,000, 7500) = 7500.$$

Функция **OptimalM**, реализующая алгоритм поиска оптимальной расстановки скобок при перемножении нескольких матриц.

Функция **OptimalM** является рекурсивной, так как она в процессе своей работы вызывает саму себя. Дно рекурсии достигается при совпадении значений двух первых параметров функции.

```
//- MultiMatrix.h
#define OPTIMALM_PARM(x) ((int*)x) // представление двумерного массива
int OptimalM(
                int i,        // [in] номер первой матрицы
                int j,        // [in] номер последней матрицы
                int n,         // [in] количество матриц
                const int c[], // [in] массив размерностей
                int* s         // [out] результат: позиции скобок
        );
```

```
// - MultiMatrix.cpp
#include "stdafx.h"
#include <memory.h>
#include "MultiMatrix.h"
#define INFINITY  0x7fffffff
int OptimalM(int i, int j, int n, const int c[], int *s)
{
 #define OPTIMALM_S(x1,x2)  (s[(x1-1)*n+x2-1])
 int o =INFINITY, bo = INFINITY;
 if (i < j)
  {
     for (int k = i; k < j;k++)
             {
        bo = OptimalM(i,k, n, c, s)+
            OptimalM(k+1,j,n, c, s)+ c[i- 1]*c[k]*c[j];
               if (bo < o){o = bo; OPTIMALM_S(i,j) = k;}
             }
  }
  else  o = 0;
 return o;
 #undef OPTIMALM_S
};
```

Расстановку скобок в заданной последовательности матриц можно осуществить с помощью двумерного массива **s**, возвращаемого функцией в последнем параметре.

```
- расстановка скобок (рекурсивное решение)

размерности матриц: (30,35) (35,15) (15,5) (5,10) (10,20) (20,25)
минимальное количество операций умножения: 15125

матрица S

0  1  1  3  3  3
0  0  2  3  3  3
0  0  0  3  3  3
0  0  0  0  4  5
0  0  0  0  0  5
0  0  0  0  0  0
Для продолжения нажмите любую клавишу . . .
```

Скобки расставляются по принципу «сначала внешние – затем внутренние». Найдем элемент $(1, 6)$ в матрице **s**. Он равен 3. Это означает, что точка разрыва находится между первой и шестой матрицей после третьей матрицы, что позволяет расставить внешние скобки следующим образом: $(A_1 \times A_2 \times A_3) \times (A_4 \times A_5 \times A_6)$.

Точку разрыва между первой и третьей матрицей определяет элемент $(1, 3)$, а между четвертой и шестой – $(4, 6)$. После расстановки скобок выражение будет выглядеть следующим образом $(A_1 \times (A_2 \times A_3)) \times ((A_4 \times A_5) \times A_6)$. Полученная расстановка скобок позволяет получить минимальное количество операций умножения, равное 15 125.

Решение задачи вычисления длины наибольшей общей подпоследовательности

Пусть $X = \{x_i\}_n$ – последовательность n символов $x_1, x_2, \ldots, x_n$. Последовательность $Z = \{z_i\}_k$ называется ***подпоследовательностью*** последовательности X, если она может быть получена из последовательности X удалением ее некоторых элементов. Например, если X – последовательность символов A, D, C, F, G, E, то последовательность Z, состоящая из символов D, F, E, является подпоследовательностью X, так как она может быть получена с помощью удаления элементов A, C, G.

Решение задачи вычисления длины наибольшей общей подпоследовательности

Подпоследовательность $Z = \{z_i\}_k$ называется ***общей подпоследовательностью*** последовательностей $X = \{x_i\}_n$ и $Y = \{y_i\}_p$, если она является подпоследовательностью обеих последовательностей. Например, подпоследовательность A, C, G является общей подпоследовательностью последовательностей A, L, D, C, F, G, E и L, A, D, C, M, G, E.

Решение задачи вычисления длины наибольшей общей подпоследовательности

Задача LCS (наибольшей общей подпоследовательности) имеет оптимальную подструктуру: проблему можно разбить на более мелкие и простые подзадачи, которые, в свою очередь, можно разбить на более простые подзадачи и так далее, пока, наконец, решение не станет тривиальным. LCS, в частности, имеет перекрывающиеся подзадачи: решения подзадач высокого уровня часто повторно используют решения подзадач более низкого уровня. Проблемы с этими двумя свойствами поддаются подходам динамического программирования, при которых решения подзадач запоминаются, то есть решения подзадач сохраняются для повторного использования.

Длиной последовательности (***подпоследовательности***) будем называть количество ее элементов. Например, длина последовательности A, D, C, F, G, E равна 6.

Подпоследовательность $Z = \{z_i\}_k$ называется ***наибольшей общей подпоследовательностью*** последовательностей $X = \{x_i\}_n$ и $Y = \{y_i\}_p$, если Z имеет наибольшую длину среди всех общих подпоследовательностей.

Для обозначения наибольшей общей подпоследовательности используется сокращение **LCS** (longest common subsequence).

Рекурсивный алгоритм вычисления длины **LCS** $Z=\{z_i\}_k$ для двух последовательностей $X=\{x_i\}_n$ и $Y=\{y_i\}_p$ основывается на трех следующих утверждениях:

1. Если $x_n=y_p$ то $z_k= x_n=y_p$ и $Z=\{z_i\}_{k-1}$ является **LCS** для $X=\{x_i\}_{n-1}$ и $Y=\{y_i\}_{p-1}$
2. Если $x_n\neq y_p$ и $z_k\neq x_n$ то $Z=\{z_i\}_k$ является **LCS** для $X=\{x_i\}_{n-1}$ и $Y=\{y_i\}_p$
3. Если $x_n\neq y_p$ и $z_k\neq y_p$, то $Z=\{z_i\}_k$ является **LCS** для $X=\{x_i\}_n$ и $Y=\{y_i\}_{p-1}$.

Обозначив через $c_{n,p}$ длину LCS для последовательностей $X=\{x_i\}_n$ и $Y=\{y_i\}_p$, можем записать следующее рекуррентное соотношение:

$$c_{n,p}=\begin{cases}0, n=0 \vee p=0,\\ c_{n-1,p-1}+1, n>0 \wedge p>0 \wedge x_n=y_p,\\ \max(c_{n,p-1}, c_{n-1,p}), n>0 \wedge p>0 \wedge x_n \neq y_p.\end{cases}$$

Рассмотрим пример вычисления длины **LCS** для последовательностей $X=A,L,D,C$ и $Y=L,A,D,C,M$. Вычисление осуществляется по шагам. Все шаги вычисления можно разбить на две группы: с 1 по 17 и с 18 по 26.

Первая группа соответствует рекурсивному погружению, вторая – восходящему вычислению.
Результатом вычисления является значение $c_{4,5}=3$ равное длине **LCS**.
Нетрудно убедиться, что для последовательностей X и Y существуют две **LCS**: L,D,C и A,D,C имеющих длину 3.

X=A,L,D,C

Y=L,A,D,C,M

$$c_{n,p} = \begin{cases} 0, n = 0 \vee p = 0, \\ c_{n-1,p-1} + 1, n > 0 \wedge p > 0 \wedge x_n = y_p, \\ \max(c_{n,p-1}, c_{n-1,p}), n > 0 \wedge p > 0 \wedge x_n \neq y_p. \end{cases}$$

1) $c_{4,5} = \max(c_{4,4}, c_{3,5});$.
2) $c_{4,4} = c_{3,3} + 1;$
3) $c_{3,5} = \max(c_{3,4}, c_{2,5});$
4) $c_{3,4} = \max(c_{3,3}, c_{2,4});$.
5) $c_{3,3} = c_{2,2} + 1;$
6) $c_{2,5} = \max(c_{2,4}, c_{1,5});$
7) $c_{2,4} = \max(c_{2,3}, c_{1,4});$
8) $c_{2,3} = \max(c_{2,2}, c_{1,3});$
9) $c_{2,2} = \max(c_{2,1}, c_{1,2});$
10) $c_{1,5} = \max(c_{1,4}, c_{0,5}) = c_{1,4};$
11) $c_{1,4} = \max(c_{1,3}, c_{0,4}) = c_{1,3};$
12) $c_{1,3} = \max(c_{1,2}, c_{0,3}) = c_{1,2};$
13) $c_{1,2} = c_{0,1} + 1 = 1;$
14) $c_{1,3} = c_{1,2} = 1;$
15. $c_{1,4} = c_{1,3} = 1;$
16. $c_{1,5} = c_{1,4} = 1;$
17. $c_{2,1} = c_{1,0} + 1 = 1;$
18. $c_{2,2} = \max(1, 1) = 1;$
19. $c_{2,3} = \max(1, 1) = 1;$
20. $c_{2,4} = \max(1, 1) = 1;$
21. $c_{2,5} = \max(1, 1) = 1;$
22. $c_{3,3} = c_{2,2} + 1 = 2;$
23. $c_{3,4} = \max(2, 1) = 2;$
24. $c_{3,5} = \max(2, 1) = 2;$
25. $c_{4,4} = c_{3,3} + 1 = 2 + 1 = 3;$
26. $c_{4,5} = \max(3, 2) = 3.$

```
// - LCS.h
// -- рекурсивное  вычисление длины LCS
int lcs (
                        int lenx,        // длина   последовательности  X
                        const char x[],   // последовательность X
                        int leny,        // длина   последовательности  Y
                        const char y[]    // последовательность Y
        );
```

```
// - LCS.cpp
// -- рекурсивное  вычисление длины LCS
#include "stdafx.h"
#include <algorithm>
#include "LCS.h"
int lcs (int lenx, const char x[],
               int leny, const char y[])
{
  int rc = 0;
  if (lenx > 0 && leny > 0)
  {
   if (x[lenx-1] == y[leny-1]) rc = 1 + lcs(lenx-1, x,leny-1, y);
   else rc = std::max(lcs(lenx, x,leny-1, y), lcs(lenx-1, x,leny, y));
  }
 return rc;       //длина LCS
}
```

```
-- вычисление длины LCS для X и Y(рекурсия)
-- последовательность X: ALDC
-- последовательность Y: LADCM
-- длина LCS: 3
Для продолжения нажмите любую клавишу . . .
```

```
// - main
// -- вычисления длины LCS
#include "stdafx.h"
#include <iostream>
#include "LCS.h"
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 char X[]="ALDC", Y[]="LADCM";
 std::cout<<std::endl<<"-- вычисление длины LCS для X и Y(рекурсия)";
 std::cout<<std::endl<<"-- последовательность X: "<< X;
 std::cout<<std::endl<<"-- последовательность Y: "<< Y;
 int s = lcs(
          sizeof(X)-1,  // длина   последовательности  X
          "ALDC",       // последовательность X
          sizeof(Y)-1,  // длина   последовательности  Y
          "LADCM"       // последовательность Y
         );
 std::cout<<std::endl<< "-- длина LCS: "<<s<<std::endl;
 system("pause");
 return 0;
}
```

Для построения наибольшей общей подпоследовательности используются 2 массива **C** и **B**.

Номера строк матриц, начиная с первой, соответствуют номерам элементов 1 последовательности, а номера столбцов, начиная с 1, - номерам элементов 2 последовательности.

Массив **C** предназначен для вычисления длины LCS. Вычисление элементов осуществляется пошагово, начиная с северо-западного угла. В результате в последней строке последнего столбца находится длина LCS.

Массив **B** предназначен для построения массива LCS.

C		B	D	C	A	B	A
	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

B		B	D	C	A	B	A
A		↑	↑	↑	↖	←	↖
B		↖	←	←	↑	↖	←
C		↑	↑	↖	←	↑	↑
B		↖	↑	↑	↑	↖	←
D		↑	↖	↑	↑	↑	↑
A		↑	↑	↑	↖	↑	↖
B		↖	↑	↑	↑	↖	↑

В матрице **B** есть 3 типа символов, обозначающих направление: TOP, LEFT и LEFTTOP.

Элементы массива выбираются с последнего элемента последней строки, используя значения как указатель направления перехода к следующему.

Выбранные элементы массива **B**, значение которых LEFTTOP, соответствуют элементам, вошедшим в LCS.